目录

一 LINUX 下的 shell 编程案例 **2**

1 程序功能说明 2

2 Shell 脚本执行方法 2

3 案例展示 3

4 Shell 程序调试方法 6

5 源代码展示 9

一 LINUX 下的 shell 编程案例

本次作业我设计并实现了一个基于 Bash Shell 的 Linux 命令答题系统。程序通过菜单提供开始答题、查看历史成绩、清空历史成绩和退出系统等功能，题目从外部题库文件中读取，答题结束后会把本次成绩保存到成绩文件中。这个案例既能练习 Linux 基础命令，也能体现 Shell 脚本中条件判断、循环控制、函数封装、文件读写和程序调试等知识点。

1 程序功能说明

本程序的主要功能如下：

- 显示主菜单，用户可以通过输入编号选择不同功能。
- 从 `questions.txt` 题库文件中逐行读取题目和标准答案。
- 判断用户输入是否为空、是否与标准答案一致，并根据结果计算得分和错误次数。
- 当答错次数达到设定上限时，系统自动结束本轮答题。
- 将每次答题的时间、得分、已答题数和错误题数保存到 `score.txt` 文件。
- 支持查看历史成绩，并使用 `awk` 统计历史最高分。
- 支持清空历史成绩，清空前需要用户再次确认，避免误操作。

程序采用“主菜单 + 功能函数”的结构，整体逻辑比较清晰。用户进入程序后，主循环会不断显示菜单，直到用户选择退出。每个功能被封装成独立函数，例如 `init_env`、`show_menu`、`start_quiz`、`save_score`、`show_history` 和 `clear_history`，这样可以减少重复代码。

2 Shell 脚本执行方法

在 Linux 平台下，可以进入脚本所在目录后执行以下命令运行程序：

```
cd task2
chmod +x linux_quiz.sh
./linux_quiz.sh
```

其中，`chmod +x linux_quiz.sh` 用于给脚本添加可执行权限，`./linux_quiz.sh` 表示执行当前目录下的脚本文件。

3 案例展示

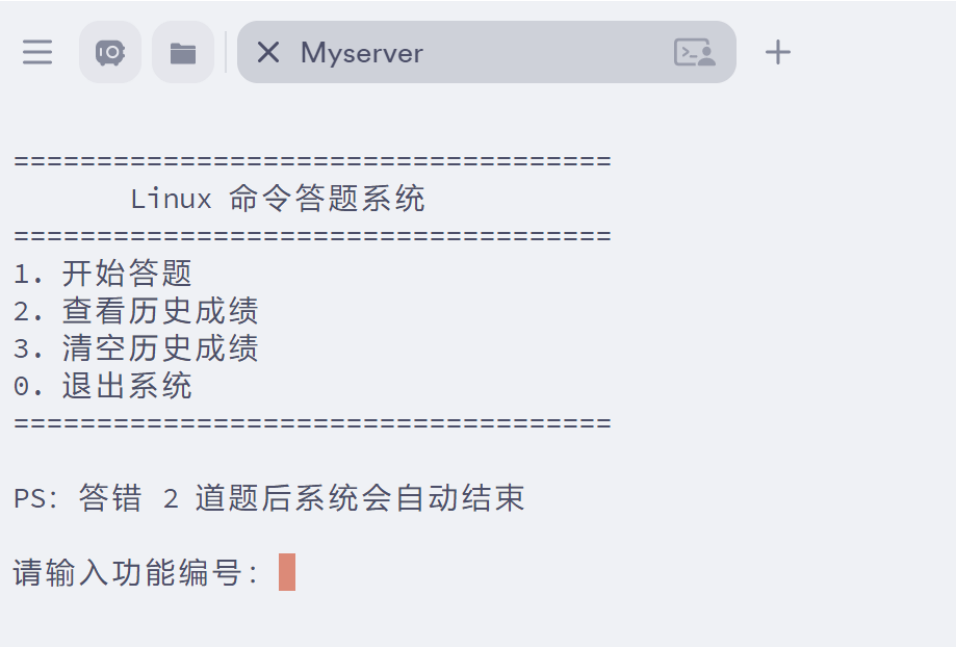
接上次的作业，这次这个也是在我的 Ubuntu 服务器上运行和测试这个 shell 文件，之后的调试也是在这台服务器上进行。

首先创建一个目录来存放本次作业。然后加入 sh 文件和 txt 题库文件。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/  clashctl/  clash-for-linux-install/  derp/  frp/  linux_homework/  n2n/  rustDesk/  snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir task2
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/     clash-for-linux-install/  frp/             n2n/       snap/
clashctl/  derp/                     linux_homework/  rustDesk/  task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh  questions.txt
```

按照之前的部分运行这个脚本。

主菜单页面：



选择 1，开始答题：

```
-----------------------------
第 1 题: 查看当前所在目录
请输入答案: pwd
回答正确, +1 分。
当前得分: 1, 错误次数: 0/2

按 Enter 继续下一题...
```

```
-----------------------------
第 5 题: 创建空文件
请输入答案: touch
回答正确, +1 分。
当前得分: 5, 错误次数: 0/2

按 Enter 继续下一题...
```

```
-----------------------------
第 11 题: 查看当前登录用户
请输入答案: whoami
回答正确, +1 分。
当前得分: 11, 错误次数: 0/2

按 Enter 继续下一题...
```

```
-----------------------------
第 22 题: 修改文件权限
请输入答案: chmod
回答正确, +1 分。
当前得分: 22, 错误次数: 0/2

按 Enter 继续下一题...
```

图 1 答题界面

答题结束界面:

```
-----------------------------------
第 11 题: 查看当前登录用户
请输入答案: uname
回答错误! 正确答案是: whoami
当前得分: 9, 错误次数: 2/2


错误次数达到 2 次, 答题结束。

========== 答题结果 ==========
本次得分: 9
已答题数: 11
错误题数: 2
成绩已保存到 ./score.txt

按 Enter 键返回主菜单...
```

图 2 答题结束界面

历史成绩界面:

```
========== 历史成绩 ==========
最高分: 24
-----------------------------------
时间 | 得分 | 已答题数 | 错误题数
-----------------------------------
2026-04-25 21:07:00 | 1 | 1 | 2
2026-04-27 13:17:50 | 9 | 11 | 2
2026-04-27 13:20:38 | 1 | 1 | 2
2026-04-27 13:29:59 | 24 | 25 | 1

按 Enter 键返回主菜单...
```

图 3 历史成绩

注意，题库和历史成绩都是保存在当前目录下的 txt 文件中。目前题库只加了 25 道题。以后如要添加可以直接在 `questions.txt` 文件中添加题目和答案，格式为“题目 | 答案”，每行一题。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  questions.txt*  score.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# cat score.txt
2026-04-25 21:07:00|1|1|2
2026-04-27 13:17:50|9|11|2
2026-04-27 13:20:38|1|1|2
2026-04-27 13:29:59|24|25|1
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# cat questions.txt
查看当前所在目录|pwd
查看当前目录下文件和文件夹|ls
切换目录|cd
创建目录|mkdir
创建空文件|touch
查看文件内容|cat
```

图 4 题库和历史 txt

清空历史，可以选择清空或者不清空。

```
====================================
      Linux 命令答题系统
====================================
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
====================================

PS: 答错 2 道题后系统会自动结束

请输入功能编号: 3

确认清空历史成绩吗? 输入 y 确认: n
已取消清空操作。

按 Enter 键返回主菜单...
```

```
====================================
      Linux 命令答题系统
====================================
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
====================================

PS: 答错 2 道题后系统会自动结束

请输入功能编号: 3

确认清空历史成绩吗? 输入 y 确认: y
历史成绩已清空。

按 Enter 键返回主菜单...
```

图 5 清空历史成绩

退出系统。

```
====================================
      Linux 命令答题系统
====================================
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
====================================

PS: 答错 2 道题后系统会自动结束

请输入功能编号: 0
系统已退出。
```

图 6 退出系统

4 Shell 程序调试方法

在开发和调试 Shell 脚本时，我主要使用 `bash -n linux_quiz.sh` 命令来检查脚本的语法错误。这个命令会解析脚本但不执行它，可以快速发现语法问题。

如果正常的话，命令行会没有输出，表示没有语法错误。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# bash -n linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2#
```

如果有错误，会显示错误信息和所在行号，方便定位问题。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# bash -n linux_quiz.sh
linux_quiz.sh: line 143: syntax error near unexpected token `}'
linux_quiz.sh: line 143: `}'
```

使用 `vim` 打开这个文件，可以看到，是这里少了一个 `fi`，导致语法错误。修复后再次运行检查后就没有错误了。

```
# 查看历史成绩，并在最上方显示最高分
show_history() {
    echo
    echo "========== 历史成绩 ==========="

    if [ ! -f "$SCORE_FILE" ]; then
        echo "暂无成绩文件。"
        return
    fi

    if [ ! -s "$SCORE_FILE" ]; then
        echo "暂无历史成绩。"
        return
```

图 7 错误修正

完成 shell 文件之后，可以使用 `bash -x linux_quiz.sh` 来执行脚本并显示每条命令的执行过程，这样可以更细致地调试程序逻辑和变量值。

```
+ show_menu
+ echo

+ echo ====================================
====================================
+ echo '       Linux 命令答题系统'
       Linux 命令答题系统
+ echo ====================================
====================================
+ echo '1. 开始答题'
1. 开始答题
+ echo '2. 查看历史成绩'
2. 查看历史成绩
+ echo '3. 清空历史成绩'
3. 清空历史成绩
+ echo '0. 退出系统'
0. 退出系统
+ echo ====================================
====================================
+ echo

+ echo 'PS: 答错 2 道题后系统会自动结束'
PS: 答错 2 道题后系统会自动结束
+ echo

+ read -r -p 请输入功能编号:  choice
请输入功能编号:
```

图 8 执行跟踪调试

另外，我还构造了一些情况来进行测试，比如删除 `questions.txt`，输入非法选项等，来验证程序的健壮性和错误处理能力。

删除 `questions.txt` 后运行程序：

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  questions.txt*  score.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# rm questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  score.txt
```

图 9 删除题库文件

```
题库文件不存在: ./questions.txt

按 Enter 键返回主菜单...
```

图 10 运行程序

可以看出程序并没有崩溃，而是提示了题库文件不存在，说明程序的错误处理机制是有效的。

恢复题库文件后，答题输入中文选项：

```
-------------------------------------
第 1 题：查看当前所在目录
请输入答案：中文
回答错误！正确答案是：pwd
当前得分：0，错误次数：1/2

按 Enter 继续下一题...
```

图 11 输入中文选项

程序能够正确识别并说明该答案错误，说明程序对输入的处理也是正确的。

以及在主菜单页面，输入非法选项，程序不会崩溃，而是继续进行循环，提示用户输入正确选项。

```
=====================================
      Linux 命令答题系统
=====================================
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
=====================================

PS：答错 2 道题后系统会自动结束

请输入功能编号：asdads
```

图 12 输入非法选项

5 源代码展示

以下是我的本次 shell 编程作业的完整源代码：

```bash
#!/usr/bin/env bash

QUESTION_FILE="./questions.txt"
SCORE_FILE="./score.txt"
```

```
WRONG_LIMIT=2

# 初始化运行环境
init_env() {
    if [ ! -f "$SCORE_FILE" ]; then
        touch "$SCORE_FILE"
    fi
}

# 暂停，便于用户查看结果
pause_screen() {
    echo
    read -r -p " 按 Enter 键返回主菜单..."
}

# 显示主菜单
show_menu() {
    echo
    echo "====================================="
    echo "       Linux 命令答题系统"
    echo "====================================="
    echo "1. 开始答题"
    echo "2. 查看历史成绩"
    echo "3. 清空历史成绩"
    echo "0. 退出系统"
    echo "====================================="
    echo
    echo "PS: 答错 $WRONG_LIMIT 道题后系统会自动结束"
    echo
}

# 核心答题功能
start_quiz() {
    score=0
    wrong=0
    total=0
    answered=0

```

```
    if [ ! -f "$QUESTION_FILE" ]; then
        echo " 题库文件不存在: $QUESTION_FILE"
        return
    fi

    # 使用文件描述符 3 读取题库，避免用户输入被题库文件提前消费。
    while IFS='|' read -r question correct_answer <&3; do
        # 跳过空行
        if [ -z "$question" ]; then
            continue
        fi

        total=$((total + 1))

        echo "-----------------------------------"
        echo " 第 $total 题: $question"
        read -r -p " 请输入答案: " user_answer

        # 去掉用户输入前后的空格
        user_answer="$(echo "$user_answer" | xargs)"

        if [ -z "$user_answer" ]; then
            echo " 答案不能为空，本题判定为错误。"
            wrong=$((wrong + 1))
        elif [ "$user_answer" = "$correct_answer" ]; then
            echo " 回答正确，+1 分。"
            score=$((score + 1))
            answered=$((answered + 1))
        else
            echo " 回答错误！正确答案是: $correct_answer"
            wrong=$((wrong + 1))
            answered=$((answered + 1))
        fi

        echo " 当前得分: $score，错误次数: $wrong/$WRONG_LIMIT"
        echo

        if [ "$wrong" -ge "$WRONG_LIMIT" ]; then
```

```
            echo
            echo " 错误次数达到 $WRONG_LIMIT 次，答题结束。"
            break
        fi

        read -r -p " 按 Enter 继续下一题..."
        clear

    done 3< "$QUESTION_FILE"

    echo
    echo "========== 答题结果 ==========="
    echo " 本次得分: $score"
    echo " 已答题数: $answered"
    echo " 错误题数: $wrong"

    save_score "$score" "$answered" "$wrong"
}

# 保存成绩
save_score() {
    current_score="$1"
    answered_count="$2"
    wrong_count="$3"
    current_time="$(date '+%Y-%m-%d %H:%M:%S')"

    echo "$current_time|$current_score|$answered_count|$wrong_count" >>
    ↪  "$SCORE_FILE"

    if [ -w "$SCORE_FILE" ]; then
        echo " 成绩已保存到 $SCORE_FILE"
    else
        echo " 成绩文件不可写，保存失败。"
    fi
}

# 查看历史成绩，并在最上方显示最高分
show_history() {
```

```
    echo
    echo "========== 历史成绩 ==========="

    if [ ! -f "$SCORE_FILE" ]; then
        echo " 暂无成绩文件。"
        return
    fi

    if [ ! -s "$SCORE_FILE" ]; then
        echo " 暂无历史成绩。"
        return
    fi

    highest_score="$(awk -F'|' 'BEGIN {max=0} {if ($2 > max) max=$2} END
    ↪ {print max}' "$SCORE_FILE")"

    echo " 最高分: $highest_score"
    echo "-----------------------------------"
    echo " 时间 | 得分 | 已答题数 | 错误题数"
    echo "-----------------------------------"

    while IFS='|' read -r time score answered wrong; do
        if [ -n "$time" ]; then
            echo "$time | $score | $answered | $wrong"
        fi
    done < "$SCORE_FILE"
}

# 清空历史成绩
clear_history() {
    echo
    read -r -p " 确认清空历史成绩吗? 输入 y 确认: " confirm

    if [ "$confirm" = "y" ] || [ "$confirm" = "Y" ]; then
        > "$SCORE_FILE"
        echo " 历史成绩已清空。"
    else
        echo " 已取消清空操作。"
```

```
    fi
}

# 主程序入口
init_env

while true; do
    clear
    show_menu
    read -r -p " 请输入功能编号: " choice

    case "$choice" in
        1)
            clear
            start_quiz
            pause_screen
            ;;
        2)
            clear
            show_history
            pause_screen
            ;;
        3)
            clear_history
            pause_screen
            ;;
        0)
            echo " 系统已退出。"
            exit 0
            ;;
        *)
            echo " 输入错误，请输入 0 到 3 之间的数字。"
            ;;
    esac
done
```